Exercícios — Alocação dinâmica de vetores e matrizes

*Os exercícios marcados com **[Formato de prova]** seguem o modelo das avaliações: resolva-os primeiro no papel, sem o computador, e só depois confira executando o código. Nos demais, execute e teste o seu programa com diferentes entradas. Em todos os programas, libere com* `delete` / `delete[]` *tudo o que foi alocado com* `new` / `new[]`.

Nível Fácil

1) Escreva um programa que aloque dinamicamente **um** inteiro com `new`, leia um valor do teclado para dentro dele, multiplique o valor por 2 usando o ponteiro, imprima o resultado e libere a memória com `delete`. Por que o ponteiro continua guardando um endereço depois do `delete`? Que valor é recomendável atribuir a ele em seguida?

2) Escreva um programa que leia a quantidade `n` de notas, aloque um vetor de `double` com `new[]` (de tamanho `n`, definido só em tempo de execução), leia as notas, imprima a média e a maior nota e libere o vetor com `delete[]`. Por que não é possível declarar `double notas[n];` de forma portável?

3) Cada trecho abaixo contém um erro de gerenciamento de memória. Identifique o erro (vazamento, ponteiro pendurado ou `delete` duplo) e mostre como corrigi-lo.

```
// a)
int *p = new int(5);
p = new int(8);
delete p;

// b)
int *q = new int[3];
delete[] q;
q[0] = 1;

// c)
int *r = new int(2);
int *s = r;
delete r;
delete s;
```

4) Escreva a função `int* criaVetor(int n)`, que aloca um vetor de `n` inteiros, preenche cada posição `i` com `i * i` e **devolve** o endereço do vetor. No `main`, leia `n`, chame a função, imprima o vetor e libere a memória. Por que é correto devolver o ponteiro de um vetor criado com `new[]`, mas é um erro devolver o endereço de um vetor declarado localmente (`int v[10];`)?

5) **[Formato de prova]** Qual é a saída impressa na tela pelo código abaixo?

```
int *p = new int(7);
int *q = p;
*q = *q * 2;
q = new int(*p + 1);
*p += *q;
cout << *p << " " << *q;
delete p;
delete q;
```

A) 14 15
B) 29 29
C) 29 15
D) 15 29
E) Erro de compilação

Nível Médio

6) Leia a quantidade `n` e `n` números reais. Armazene-os em um vetor alocado dinamicamente e imprima-os na **ordem inversa**. Em seguida, crie um **segundo** vetor dinâmico com o mesmo tamanho contendo o quadrado de cada valor, imprima-o e libere os dois vetores.

7) Escreva um programa que leia números inteiros até que o usuário digite 0, **sem saber** de antemão quantos serão. Use um vetor dinâmico que cresce **uma posição por vez**: a cada novo valor, aloque um vetor com uma posição a mais, copie os valores antigos, libere o vetor antigo com `delete[]` e passe a usar o novo. Ao final, imprima os valores e a quantidade lida.

8) Escreva as funções `int** criaMatriz(int L, int C)` (aloca uma matriz **linha por linha**) e `void liberaMatriz(int **m, int L)`. No `main`, leia `L` e `C`, crie a matriz, preencha-a com valores lidos, imprima a soma de cada linha e a soma de cada coluna e libere a matriz. Qual é a ordem correta da liberação: primeiro as linhas ou primeiro o vetor de ponteiros? Por quê?

9) Considere a struct abaixo. Leia a quantidade `n` de produtos, aloque dinamicamente um vetor de `n` structs `Produto`, leia os dados de cada um, imprima o produto **mais caro** e o valor total do estoque (soma de `preco * qtd`). Acesse os campos por meio de ponteiro, com o operador `->`, em ao menos uma parte do programa. Libere a memória ao final.

```
struct Produto {
    string nome;
    float preco;
    int qtd;
};
```

10) Escreva uma função **template** `T* criaVetor(int n, T valor)`, que aloca um vetor dinâmico de `n` elementos do tipo `T`, preenche todas as posições com `valor` e devolve o ponteiro. No `main`, use-a para criar um vetor de 5 `int` com o valor 7, um de 3 `double` com 1.5 e um de 4 `string` com "ok". Imprima os três vetores e libere cada um.

11) **[Formato de prova]** Qual é a saída impressa na tela pelo código abaixo?

```
int *v = new int[4];
for (int i = 0; i < 4; i++)
    v[i] = i + 1;
int *p = v + 2;
p--;
*p = *p + *(p + 2);
cout << v[1] + v[3];
delete[] v;
```

A) 6
B) 10
C) 8
D) 12
E) 4

Nível Difícil

12) Escreva a função `int** transposta(int **m, int L, int C)`, que recebe uma matriz `L x C` alocada dinamicamente e **devolve uma nova matriz** `C x L` (também dinâmica) com a transposta. No `main`, leia `L`, `C` e a matriz, chame a função, imprima as duas matrizes e libere ambas (use a função `liberaMatriz`, indicando com cuidado o número de linhas de cada uma).

13) Uma matriz também pode ser alocada em **um único bloco** de `L * C` inteiros, com um vetor de ponteiros apontando para o início de cada linha. Escreva o código que aloca a matriz dessa forma (`m[0] = new int[L * C];` e `m[i] = m[i - 1] + C;`), preenche com valores lidos, imprime a diagonal principal e libera a memória. Quantos `delete[]` são necessários agora, e quantos eram necessários na alocação linha por linha? Qual é a vantagem desta forma?

14) Escreva a função `void redimensiona(int *&v, int &n, int novoN)`, que troca o vetor dinâmico `v` por um de tamanho `novoN`, copiando os `min(n, novoN)` primeiros valores, preenchendo as posições novas com 0, liberando o vetor antigo e atualizando `n`. Explique por que `v` precisa ser passado por **referência** (`int *&v`). No `main`, crie um vetor de 5 elementos, aumente-o para 8 e diminua-o para 3, imprimindo o conteúdo a cada passo.

15) O arquivo `alunos.txt` contém, na primeira linha, a quantidade `n` de alunos e, nas `n` linhas seguintes, o nome (sem espaços) e a nota de cada um. Usando a struct `Aluno` abaixo, leia `n`, aloque dinamicamente o vetor de alunos, leia os dados do arquivo, ordene o vetor da maior para a menor nota e grave em `ranking.txt` a posição, o nome e a nota de cada aluno. Verifique se o arquivo foi aberto e libere a memória no final.

```
struct Aluno {
    string nome;
    float nota;
};
```

16) **[Formato de prova]** Qual é a saída impressa na tela pelo código abaixo?

```
void f(int *&p) {
    int *n = new int[3];
    for (int i = 0; i < 3; i++)
        n[i] = p[i] * 2;
    delete[] p;
    p = n;
}

int main() {
    int *v = new int[3]{1, 2, 3};
    f(v);
    f(v);
    cout << v[0] << " " << v[1] << " " << v[2];
    delete[] v;
    return 0;
}
```

A) 2 4 6
B) 4 8 12
C) 1 2 3
D) 4 8 6
E) Erro de compilação

17) **[Formato de prova]** (1,0 ponto) Escreva uma função **template** `inverso`, que receba um vetor `v` de elementos de tipo genérico `T` e a quantidade `n` de elementos e **devolva um novo vetor dinâmico** com os mesmos elementos em ordem inversa (o vetor original não deve ser alterado). Escreva apenas a função, sem o `main`. Em seguida, diga o que o `main` abaixo imprime.

```
int main() {
    int v[5] = {5, 4, 3, 2, 1};
    char w[3] = {'c', 'b', 'a'};
    int *a = inverso(v, 5);
    char *b = inverso(w, 3);
    for (int i = 0; i < 5; i++)
        cout << a[i] << " ";
    cout << "\n";
    for (int i = 0; i < 3; i++)
        cout << b[i];
    delete[] a;
    delete[] b;
    return 0;
}
```